



VETRA

PUBLICATION SERIES • DOCUMENT 04

DOC ID: PUB-VETRA-DOC-04
MODULE: CLOUD & INFRASTRUCTURE
DATE: AUGUST 2026

• CLOUD ARCHITECTURE & PRODUCTION ENGINEERING MONOGRAPH

VETRA

Cloud, Infrastructure & Production Engineering

AWS Architecture • Terraform Infrastructure as Code • Container Deployment •
Zero-Trust Security • Production Reliability

ARCHITECTED & ENGINEERED BY

Om Rajput

B.Tech in Computer Science & Engineering (Data Science)
SVKM's Institute of Technology, Dhule

Cloud Infrastructure Framework

AWS ap-south-1 (Mumbai) • Terraform Modules • ECS Fargate • RDS
PostgreSQL 15 • Redis 7.1 • OIDC CI/CD

Staging Verification Status

Live Verified on api.vetra.dpdns.org • 5/5 Automated
Health Smoke Tests Passing

01 — CLOUD ENGINEERING PHILOSOPHY

Infrastructure Principles & Design Philosophy

Architecting veterinary operations for high availability, zero-trust network isolation, and immutable automation.

"Cloud infrastructure for rural and agrarian medicine must satisfy two competing requirements: extreme cost efficiency during development and enterprise-grade resilience during epidemiological outbreaks. In Vetra, infrastructure is treated strictly as reproducible software code. We enforce complete separation between staging and production, eliminate shared database access, and mandate automated cryptographic zero-trust boundaries across every network hop."

— Om Rajput, Cloud Architect

1. Declarative IaC

Every AWS resource (VPC, ECS, RDS, Redis, IAM) is managed through modular Terraform scripts with remote S3 state and DynamoDB locking.

2. Zero-Trust Isolation

Three-tier subnet segmentation: Public (ALB/NAT), Private (Fargate tasks), and Isolated (RDS/Redis with zero public internet routing).

3. Immutable Releases

Container images are built via multi-stage Docker builds, tagged immutably with Git commit SHAs, and deployed with zero downtime rolling updates.

Declarative IaC

Terraform 1.5+ Modules
S3 State & DynamoDB Lock
Environment Stratification
terraform apply



3-Tier Network Isolation

Public: ALB & NAT
Private: ECS Compute
Isolated: RDS & Redis
Zero Internet on Data Tier



Production Reliability

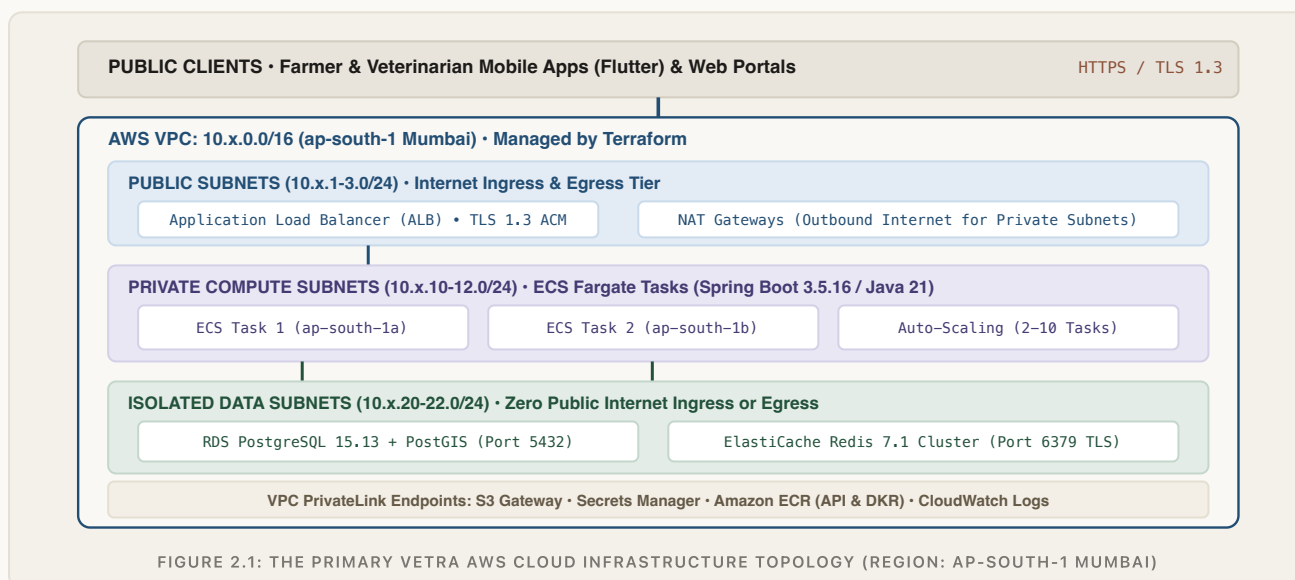
99.9% Uptime Target
Automated Multi-AZ Failover
CloudWatch 12 Alarms & SNS
Enterprise Operations

FIGURE 1.1: CORE INFRASTRUCTURE DESIGN & RELIABILITY ARCHITECTURE PRINCIPLES

02 — CLOUD TOPOLOGY

End-to-End AWS Cloud Architecture

Multi-tier regional topology in ap-south-1 (Mumbai) connecting ingress, compute, data, and telemetry.



Strict Subnet Isolation

Database and cache instances reside in isolated subnets with no route tables to internet gateways. All outbound traffic from ECS compute is routed through managed NAT Gateways.

PrivateLink VPC Endpoints

Container image pulls from ECR, secret lookups from Secrets Manager, and log streams to CloudWatch traverse AWS internal network backbones without crossing the public internet.

03 — CLOUD BUILDING BLOCKS

AWS Services & Architectural Responsibilities

Verified inventory of Amazon Web Services configured across the Vetra infrastructure modules.

AWS SERVICE	CONFIGURATION & SIZING	ARCHITECTURAL ROLE & RESPONSIBILITY	IAC MODULE
Amazon VPC	10.x.0.0/16, 6–9 Subnets, Flow Logs	Network boundaries, subnet isolation, and route table management in ap-south-1.	MODULES/VPC
AWS ALB & ACM	Internet-facing, Port 80 → 443 301 Redirect	TLS 1.3 termination, SSL offloading, and HTTP health-check routing to ECS tasks.	MODULES/ECS
AWS ECS Fargate	0.5–1.0 vCPU / 1–2 GB RAM, Rolling Deploy	Serverless container runtime executing Spring Boot 3.5.16 modular monolith.	MODULES/ECS
Amazon RDS PostgreSQL	PostgreSQL 15.13 + PostGIS, gp3 Storage	Authoritative relational database with spatial indexing for epidemiological tracking.	MODULES/RDS
Amazon ElastiCache	Redis 7.1, in-transit TLS, Auth Token	Distributed caching (passports/lookups), rate limiting, and single-flight locks.	MODULES/REDIS
Amazon ECR	Scan on push, Lifecycle Policies	Private Docker container registry storing immutable Git SHA-tagged images.	MODULES/ECR
AWS Secrets Manager	KMS Encryption at rest, PrivateLink	Secure storage and rotation for database credentials, JWT secrets, and API tokens.	MODULES/IAM
Amazon CloudWatch	12 Metric Alarms, Log Groups, SNS	Container metrics, error rate monitoring, threshold alerting, and audit log streams.	MODULES/MONITORING

Zero Unmanaged Infrastructure

No manual AWS console resource creation is permitted. Every VPC subnet, security group, and ECS parameter is declared in Terraform version-controlled repositories.

Native AWS Security Integration

All data at rest in RDS and S3 is encrypted with AWS KMS keys. Network traffic between ALB, ECS, RDS, and Redis is encrypted with TLS 1.3 in-transit.

04 — ENVIRONMENT ISOLATION

Environment Architecture: Local, Staging & Production

Strict three-stage isolation protecting production data and optimizing cloud infrastructure costs.

1. Local Development

- **Runtime:** Docker Compose.
- **Databases:** Local PostgreSQL 15 + Redis 7.
- **Cost:** \$0.00 / developer workstation.
- **Purpose:** Fast inner-loop feature development and unit testing.

2. AWS Staging (Verified)

- **CIDR:** 10.1.0.0/16 (2 AZs).
- **Compute:** 1 Fargate Task (0.5 vCPU / 1GB).
- **NAT:** Single shared NAT gateway.
- **Status:** 🟢 Live at api.vetra.dpdns.org.

3. AWS Production (Drafted)

- **CIDR:** 10.0.0.0/16 (3 AZs: a, b, c).
- **Compute:** 2–10 Fargate Tasks (1.0 vCPU / 2GB).
- **NAT:** 3 Dedicated NATs (1 per AZ).
- **Status:** **DRAFTED HA SPEC**.



05 — TERRAFORM ARCHITECTURE

Infrastructure as Code: Modular Terraform Engine

Reusable infrastructure components, state locking, and strict separation between staging and production.

Terraform Module Hierarchy

- infra/modules/vpc: VPC, public/private/isolated subnets, NAT/IGW.
- infra/modules/ecs: ECS cluster, Fargate task def, ALB, target groups.
- infra/modules/rds: RDS PostgreSQL 15, subnet groups, KMS keys.
- infra/modules/redis: ElastiCache Redis 7.1 with TLS.
- infra/modules/monitoring: CloudWatch 12 alarms, log groups, SNS.

Remote State & Concurrency Locking

Terraform state is stored in an encrypted Amazon S3 bucket with versioning enabled. State locks are acquired via Amazon DynamoDB table (vetra-terraform-locks) to prevent concurrent executions from corrupting cloud infrastructure state.

Representative Terraform Module Instantiation (infra/environments/staging/main.tf)

```
module "vpc" { source = "../../modules/vpc" environment = "staging" project = "vetra" aws_region = "ap-south-1"
vpc_cidr = "10.1.0.0/16" availability_zones = ["ap-south-1a", "ap-south-1b"] public_subnet_cidrs = ["10.1.1.0/24",
"10.1.2.0/24"] private_subnet_cidrs= ["10.1.10.0/24", "10.1.11.0/24"] isolated_subnet_cidrs=["10.1.20.0/24",
"10.1.21.0/24"] enable_nat_gateway = true single_nat_gateway = true # Cost optimization for staging }
```

Environment Parameterization

Modules expose variables for CIDR ranges, instance types, task counts, and retention days, allowing identical module logic to power both staging and production.

Accidental Deletion Prevention

Database and storage modules utilize `prevent_destroy = true` lifecycle rules and automated final snapshots to protect clinical data against accidental destruction.

06 — SUBNET SEGMENTATION

Network Architecture & Subnet Topology

Three-tier VPC subnet segmentation, routing tables, and internet gateway boundaries in ap-south-1.

Tier 1: Public Subnets

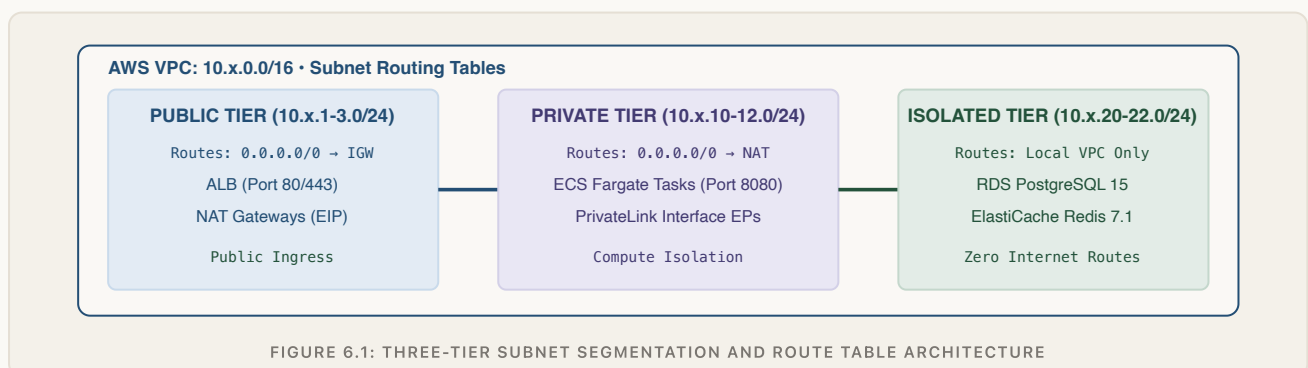
Attached to Internet Gateway. Hosts Application Load Balancers and NAT Gateways. Direct public internet ingress on ports 80 and 443 only.

Tier 2: Private Subnets

Hosts ECS Fargate container tasks. Inbound traffic allowed strictly from ALB security group. Outbound internet access via NAT Gateway.

Tier 3: Isolated Subnets

Hosts RDS PostgreSQL and ElastiCache Redis. No route to Internet Gateway or NAT. Inbound access restricted strictly to ECS security group.

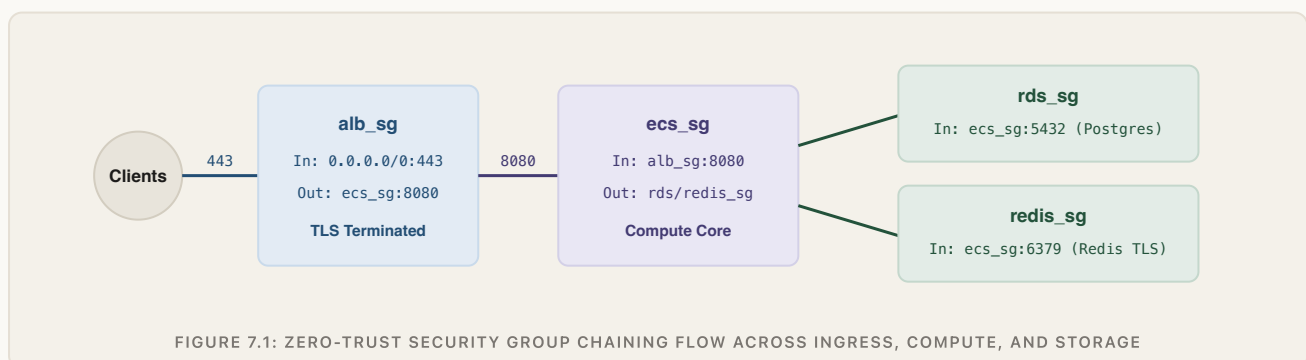


07 — DEFENSE-IN-DEPTH

Security Group Chaining & Defense-in-Depth

Chained firewall rules allowing ingress strictly from specific upstream security group IDs.

SECURITY GROUP	ALLOWED INGRESS SOURCE	PORT / PROTOCOL	SECURITY RATIONALE
vetra-alb-sg	0.0.0.0/0	TCP 80 & 443	Public entry point for HTTPS traffic and HTTP-to-HTTPS redirect.
vetra-ecs-sg	vetra-alb-sg (Only)	TCP 8080	Direct internet access blocked; tasks accept traffic strictly from ALB.
vetra-rds-sg	vetra-ecs-sg (Only)	TCP 5432	PostgreSQL database access restricted exclusively to backend tasks.
vetra-redis-sg	vetra-ecs-sg (Only)	TCP 6379	ElastiCache Redis access restricted exclusively to backend tasks.



08 — CONTAINERIZATION

Application Containerization & Multi-Stage Dockerfile

Hardened minimal alpine image runtime, non-root user execution, and container JVM optimization.

Multi-Stage Build Pipeline

- **Stage 1 (Builder):** eclipse-temurin:21-jdk-alpine resolves Maven dependencies and compiles the Spring Boot fat JAR.
- **Stage 2 (Runtime):** eclipse-temurin:21-jre-alpine contains strictly the lightweight JRE, reducing attack surface and image size to ~180MB.

Runtime Security Hardening

- **Non-Root Execution:** Runs under dedicated unprivileged user spring:spring (UID 1001).
- **Signal Handling:** Uses dumb-init to properly forward SIGTERM signals for graceful Spring Boot shutdown.
- **JVM Flags:** -XX:MaxRAMPercentage=75.0 for container memory elasticity.

Hardened Production Dockerfile Architecture

```
# — Stage 1: Build & Package — FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /workspace
COPY pom.xml .
RUN mvn dependency:go-offline -B
COPY src ./src
RUN mvn clean package -DskipTests -B
# — Stage 2: Hardened Runtime — FROM eclipse-temurin:21-jre-alpine
RUN apk add --no-cache dumb-init && addgroup -g 1001 -S spring && adduser -u 1001 -S spring -G spring
WORKDIR /app
COPY --from=builder --chown=spring:spring /workspace/target/*.jar app.jar
USER spring:spring
EXPOSE 8080
ENTRYPOINT ["/usr/bin/dumb-init", "--", "java", "-XX:MaxRAMPercentage=75.0", "-jar", "app.jar"]
```

Vulnerability Scanning on Push

Amazon ECR automatically scans container images on push (scan_on_push = true). Any Critical or High CVEs in base alpine packages are flagged before container deployment.

09 — COMPUTE MANAGEMENT

ECS Fargate Serverless Compute & Task Lifecycle

Zero-maintenance host management, Secrets Manager injection, and zero-downtime rolling deploys.

ECS Fargate Advantages

- **Zero OS Patching:** AWS manages underlying EC2 instances and hypervisor security.
- **Per-Task ENI Isolation:** Each task receives its own Elastic Network Interface (awsvpc mode) with dedicated private IP.
- **Right-Sized Capacity:** 0.5–1.0 vCPU and 1–2 GB RAM tailored to JVM memory allocations.

Rolling Deployments

- **Minimum Healthy:** `minimumHealthyPercent = 100` guarantees that existing tasks serve traffic while new tasks launch.
- **Maximum Surge:** `maximumPercent = 200` launches new task revisions in parallel before draining old revisions.

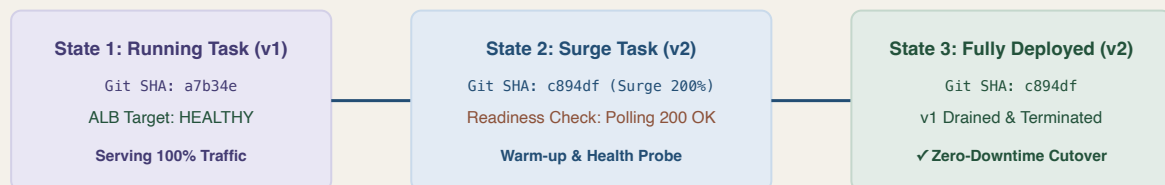
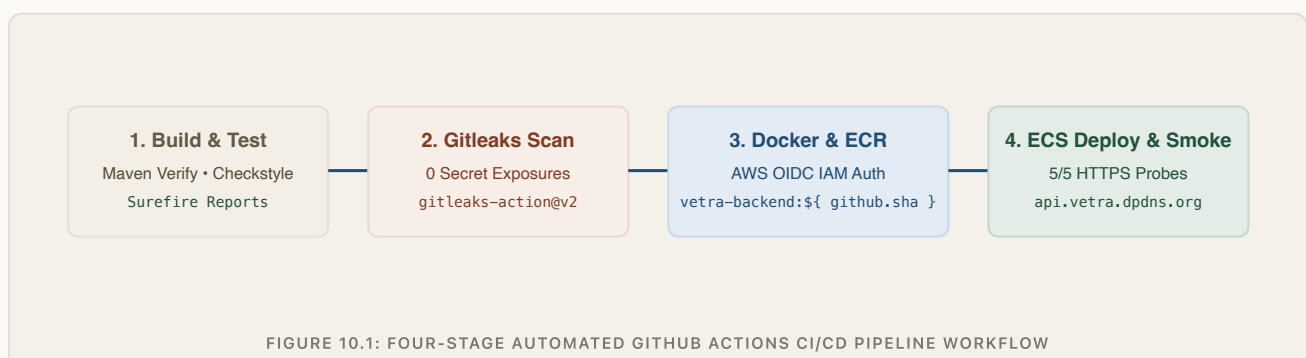
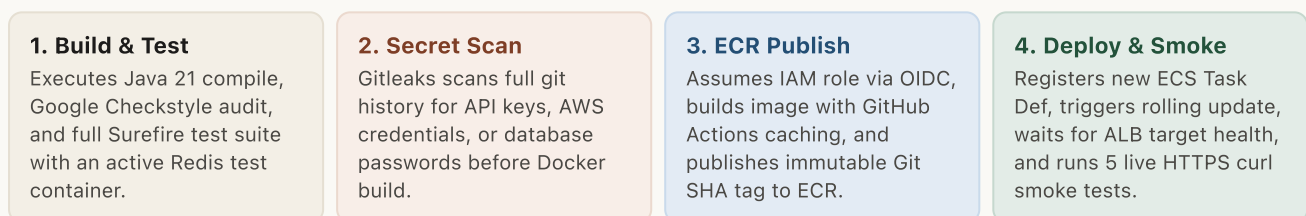


FIGURE 9.1: ZERO-DOWNTIME ROLLING DEPLOYMENT AND TASK DRAINING PROGRESSION

10 — PIPELINE AUTOMATION

CI/CD Pipeline Engineering: Code to Staging

Automated build verification, secret leak scanning, container publishing, and live smoke tests.



11 — PERSISTENCE LAYER

Database & Data Infrastructure: RDS PostgreSQL 15

Spatial database engine, HikariCP connection management, and automated KMS-encrypted backups.

RDS PostgreSQL Configuration

- **Engine:** PostgreSQL 15.13 with native postgis and uuid-oss extensions.
- **Subnet Placement:** Dedicated isolated DB subnet group across 2–3 AZs with zero internet routes.
- **Storage:** General Purpose SSD (gp3) with automated storage autoscaling and KMS encryption at rest.

Connection Pool Sizing (HikariCP)

- **Maximum Pool:** maximum-pool-size: 20 per ECS task.
- **Idle Pool:** minimum-idle: 5.
- **Timeout:** connection-timeout: 30000ms.
- **Telemetry:** HikariCP metrics instrumented via Micrometer and exported to CloudWatch.

Flyway Schema Migration Sequence (V1 to V14)

Schema migrations execute deterministically at application boot before accepting HTTP traffic. In staging and production, Flyway applies versioned SQL scripts (V1__extensions.sql through V14__refresh_tokens.sql), ensuring zero schema drift across environments.

Automated Backup Strategy

Daily automated snapshots retained for 1–7 days with Point-in-Time Recovery (PITR) enabled. Final snapshots are mandated on teardown.

Multi-AZ Standby Replication (Production)

Synchronous block-level replication to a standby instance in an alternate AZ with automated DNS failover completed in under 60 seconds.

12 — IN-MEMORY TIER

Caching & Performance Infrastructure: Redis 7.1

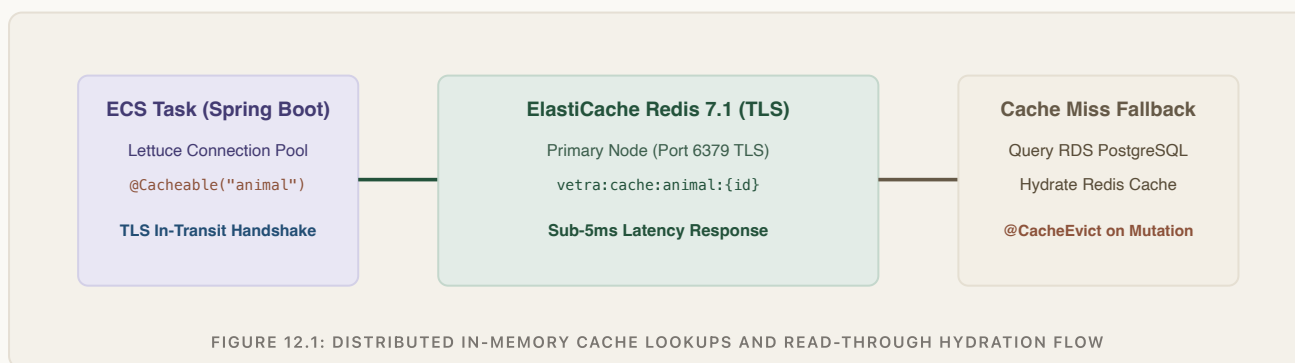
In-transit TLS encryption, deterministic eviction policies, and Lettuce connection pooling.

ElastiCache Redis Role

- **Domain Caching:** Animal passports (30m TTL), vet profiles (1h TTL), dashboard summaries.
- **Token Revocation:** Blacklisted JWTs and active refresh token rotation tracking.
- **AI Single-Flight:** Mutex locks preventing duplicate simultaneous LLM invocations.

In-Transit Encryption & Auth

- **TLS Encryption:** `transit_encryption_enabled = true` mandates TLS across all client-to-node TCP traffic.
- **AUTH Token:** Dynamic 32-character auth token injected from AWS Secrets Manager at container runtime.



13 — ZERO-TRUST SECURITY

Infrastructure Security & Keyless OIDC IAM

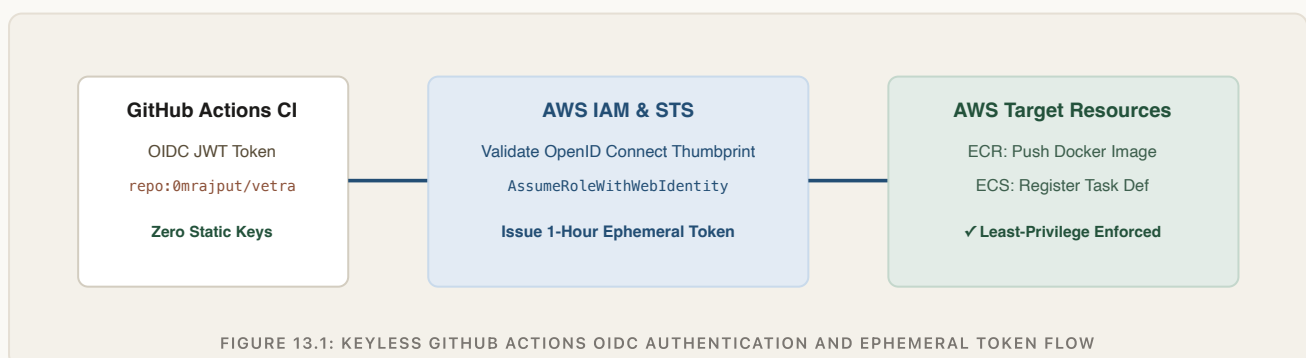
Eliminating long-lived credentials via GitHub Actions OIDC and AWS Secrets Manager PrivateLink.

Keyless GitHub Actions OIDC

Vetra uses OpenID Connect (OIDC) federation between GitHub Actions and AWS IAM. No static AWS Access Keys or Secrets are stored in GitHub repository settings. Ephemeral STS credentials valid for 1 hour are granted strictly to refs/heads/main.

IAM Role Separation

- **Task Execution Role:** Pulls ECR images and resolves Secrets Manager values.
- **Task Role:** Grants running Spring Boot application least-privilege permissions to S3 buckets and CloudWatch metrics.



14 — CLOUD VISIBILITY

Observability, CloudWatch & 12 Metric Alarms

Automated threshold alerting, SNS dispatch, and Spring Boot Actuator health endpoints.

ALARM IDENTIFIER	METRIC & THRESHOLD	IMPACT & TRIGGER CONDITION	ACTION
alb-5xx-high-rate	HTTPCode_ELB_5XX > 5 / 1min	Application fatal exceptions or upstream routing failure.	SNS P1 ALERT
alb-target-response-time	TargetResponseTime > 2.0s	Degraded latency across backend clinical endpoints.	SNS P2 ALERT
ecs-cpu-high-utilization	CPUUtilization > 80% (2 periods)	Heavy compute load; triggers task autoscaling.	AUTOSCALE
ecs-memory-high-utilization	MemoryUtilization > 80% (2 periods)	High heap consumption; potential leak indicator.	SNS ALERT
rds-cpu-high-utilization	CPUUtilization > 80%	Complex PostGIS geospatial query bottlenecks.	SNS P2 ALERT
rds-database-connections	DatabaseConnections > 80% pool	HikariCP connection pool saturation.	SNS P1 ALERT

Spring Boot Actuator Probes

- /actuator/health/liveness: Validates JVM process health.
- /actuator/health/readiness: Validates PostgreSQL and Redis connectivity before ALB targets receive traffic.

Distributed Tracing Integration

W3C traceparent context headers propagate from ALB into Spring Boot, creating distributed spans across database queries and external AI gateway calls.

15 — LIVE VALIDATION

Staging Verification & Live Operational Proof

Empirical evidence from live deployments and automated health checks on `api.vetra.dpdns.org`.

Verified Live Staging Endpoint: `https://api.vetra.dpdns.org`

AWS Staging environment successfully provisioned in `ap-south-1` via Terraform and continuously verified on every merge to main.

VERIFICATION PROBE	TARGET URI / PARAMETER	EXPECTED & OBSERVED RESULT	STATUS
HTTP → HTTPS 301 Redirect	<code>http://api.vetra.dpdns.org</code>	HTTP 301 Moved Permanently → HTTPS	PASSED
Actuator Liveness Probe	<code>/actuator/health/liveness</code>	HTTP 200 OK ("status": "UP")	PASSED
Actuator Readiness Probe	<code>/actuator/health/readiness</code>	HTTP 200 OK (DB + Redis TLS connected)	PASSED
Application Liveness	<code>/liveness</code>	HTTP 200 OK	PASSED
Application Readiness	<code>/readiness</code>	HTTP 200 OK	PASSED
Deployment Immutability	ECS Running Task Image	Matches Git Commit SHA exactly	VERIFIED

Continuous Verification

The deployment pipeline runs these 5 probes sequentially. If any probe returns non-200 or fails the TLS redirect assertion, the pipeline fails immediately and alerts the developer.

Zero Security Failures

Gitleaks secret audits run prior to every staging deployment, ensuring 0 exposed AWS secrets, private keys, or credentials exist in the Git repository.

16 — PRODUCTION DESIGN

Production Architecture & High-Availability Safeguards

Multi-AZ elasticity, auto-scaling policies, and operational safety guardrails.

Production Status: [DRAFTED HA SPEC – NOT YET APPLIED]

The production Terraform configuration in `infra/environments/production/` is completely drafted and validated for high availability. It is strictly preserved unapplied to protect cloud operational budgets until production launch review.

1. 3-AZ Multi-AZ RDS

db.t4g.medium with 100GB gp3 storage and synchronous Multi-AZ standby replication across 3 availability zones (a, b, c).

2. 3 Dedicated NATs

One NAT Gateway per AZ eliminates cross-AZ failure dependencies, guaranteeing outbound connectivity even if one AWS AZ experiences an outage.

3. Elastic Auto-Scaling

Target Tracking auto-scaling policies scale ECS Fargate tasks between 2 and 10 instances based on CPU utilization (>70%) and ALB request counts.

Disaster Recovery (RPO / RTO)

- **Recovery Point Objective (RPO):** < 5 minutes via continuous automated RDS transaction log backups.
- **Recovery Time Objective (RTO):** < 60 seconds via automated Multi-AZ standby failover.

Engineering Safety Protocols

Production applies require multi-engineer code review, isolated IAM deployment roles, and explicit terraform plan diff validation before execution.



ENGINEERING PUBLICATION SERIES • DOCUMENT 04

VETRA: Cloud & Production Engineering

AWS Architecture • Deployment • Security • Reliability

Built & Engineered by Om Rajput

B.Tech in Computer Science & Engineering (Data Science)

SVKM's Institute of Technology, Dhule

"Architecting resilient, zero-trust cloud infrastructure for mission-critical veterinary operations."